NVMe Host Accelerator v1.0

LogiCORE IP Product Guide

Vivado Design Suite

PG328 (v1.0) May 10, 2023

AMD Adaptive Computing is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this link for more information.



Table of Contents

Chapter 1: Introduction **4**
- Features 4
- IP Facts 5

Chapter 2: Overview **6**
- Core Overview 6
- Licensing and Ordering 6

Chapter 3: Product Specification **8**
- Performance and Resource Use 14
- Port Descriptions 14
- Register Space 17

Chapter 4: Designing with the Core **24**
- General Design Guidelines 24

Chapter 5: Design Flow Steps **26**
- Customizing and Generating the Core 26
- NVMe Host Accelerator Programming Sequence 30
- Constraining the Core 31
- Simulation 32
- Synthesis and Implementation 32

Chapter 6: Example Design **33**

Appendix A: Debugging **36**
- Finding Help with AMD Adaptive Computing Solutions 36
- Debug Tools 37

Appendix B: Additional Resources and Legal Notices **39**
- Finding Additional Documentation 39
- Support Resources 40
- References 40

Revision History..... 40
Please Read: Important Legal Notices..... 41

Introduction

The AMD NVMe™ Host Accelerator LogiCORE™ IP provides a simple and efficient interface to multiple NVMe drives, thereby offloading the CPU for the I/O queues and enabling a high throughput storage solution inside an FPGA. The core provides a path for either (or both) software or hardware module(s) to interface with it. Standard memory mapped AXI interfaces and AXI4-Stream interfaces allow for easy integration. The NVMe Host Accelerator core is fully parameterizable and offers multiple customizations for a resource efficient implementation tailored according to your requirements. The admin queue is managed by software (SW) and the IP offloads the following functions from the CPU:

- Submission Queue (SQ) doorbell management across multiple queues
- Completion Queue (CQ) doorbell management across multiple queues
- Building NVMe specification compliant submission queue command entry
- Completion queue entry parsing

Features

The features of the IP are categorized into Application Interface features and SSD Side Interface features.

Application Interface

- Support for AXI4 mapped slave to interface with software along with AXI4-Lite interface for register access.
- Support for AXI4-Stream to interface with hardware design modules.
- Support for configurable number of SQs per SSD (independent number of queues for hardware interface and software interface).
- SQs written into a FIFO-like interface for both memory mapped and streaming ports.
- Independently configurable depth of SQ depth on software and hardware interfaces.
- Completion entries for memory mapped interface communicated using interrupt.
- Completion entries for streaming interface communicated by generating streaming data.

SSD Side Interface

- Support for configurable number of backend SSDs.
- Round-robin arbitration of all SQs to SSD.
- AXI4 mapped master to ring SQ tail doorbells and CQ head doorbells.
- AXI4 mapped slave to read from SQs and write to CQs.

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family[1]	AMD UltraScale+™, AMD UltraScale™, and AMD Versal™ Adaptive SoC
Supported User Interfaces	AXI4-Lite, AXI4-Stream, and AXI4-Memory Mapped
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Encrypted RTL
Example Design	Verilog
Test Bench	Not Provided
Constraints File	Xilinx Constraints File
Simulation Model	Not provided
Supported S/W Driver[2]	Linux
Tested Design Flows[2]	
Design Entry	AMD Vivado™ Design Suite
Simulation	For supported simulators, see the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing
Synthesis	Not Provided
Support	
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, see the AMD Vivado™ IP catalog.
2. For the supported versions of the tools, see the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing.

Chapter 2

Overview

Core Overview

The NVMe Host Accelerator core manages the control path of multiple connected backend drives. The SSDs are responsible for pushing and pulling data from the respective buffers as provided in the SQ entry command. The data path does not pass through the IP. The NVMe Host Accelerator has configurable software and hardware access mechanisms. The design can be configured to enable or disable either of the application interfaces (software or hardware). The number of backend NVMe drives and the number of SQs per drive, per interface are all configurable. The NVMe Host Accelerator assumes a 1:1 mapping between the SQs and the CQs.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided under the terms of the Core License Agreement. The module is shipped as part of the AMD Vivado™ Design Suite. For full access to all core functionalities in simulation and in hardware, you must purchase a license for the core. To generate a full license, visit the product licensing web page. Evaluation licenses and hardware timeout licenses might be available for this core. Contact your local sales representative for information about pricing and availability.

***Note*:** To verify that you need a license, check the License column of the IP Catalog. Included means that a license is included with the AMD Vivado™ Design Suite; Purchase means that you have to purchase a license to use the core.

For more information about this core, visit the NVMe Host Accelerator product web page.

Information about other AMD LogiCORE™ IP modules is available at the Intellectual Property page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your local sales representative.

License Checkers

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

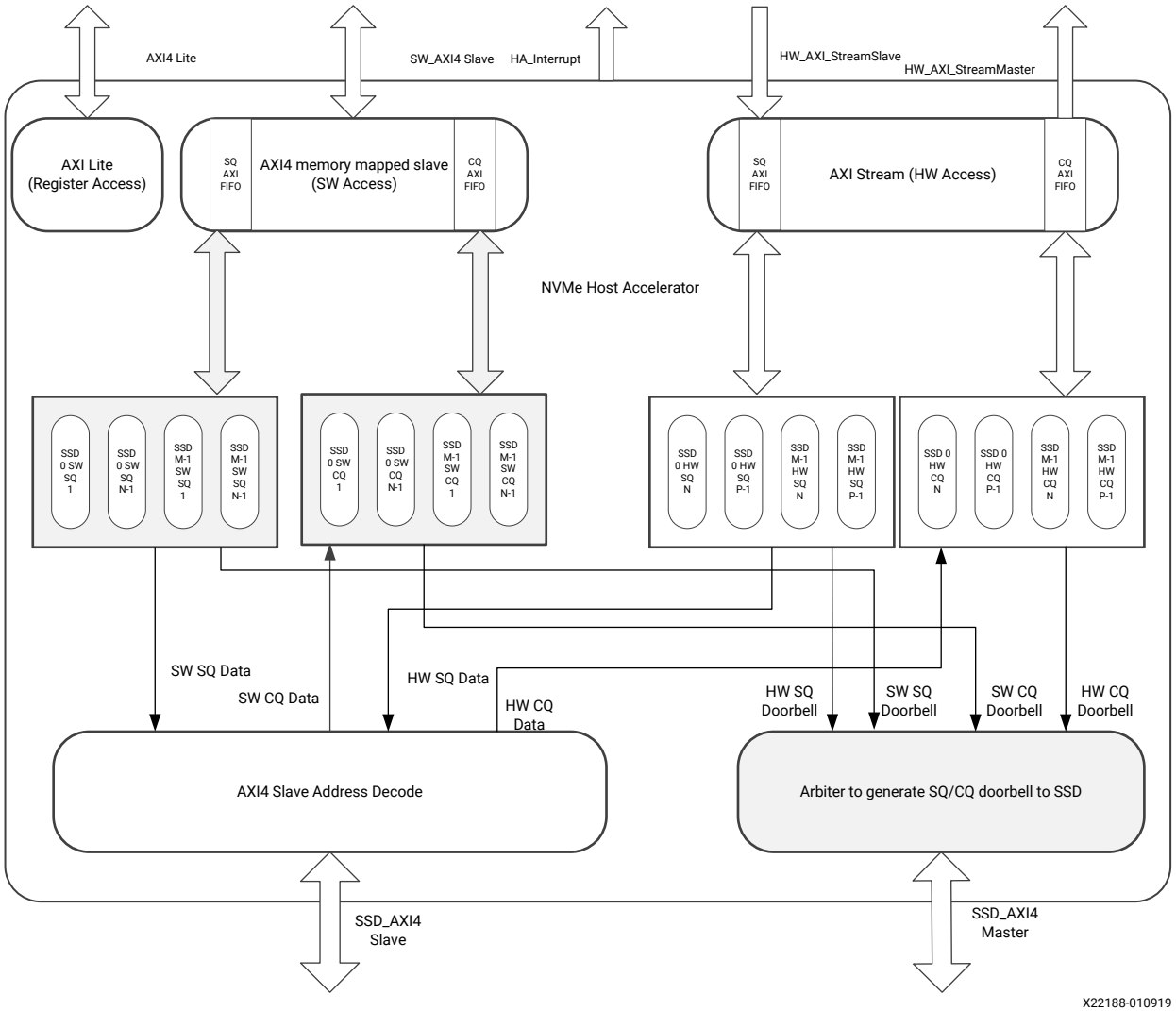
- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)

IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Product Specification

The following figure shows the top level block diagram of the IP exposing all the relevant interfaces. The software interface to the IP is via AXI4 and AXI4-Lite. It supports XDMA in UltraScale+, QDMA in AMD Versal™ adaptive SoC.

Figure 1: **NVMe Host Accelerator Block Diagram**

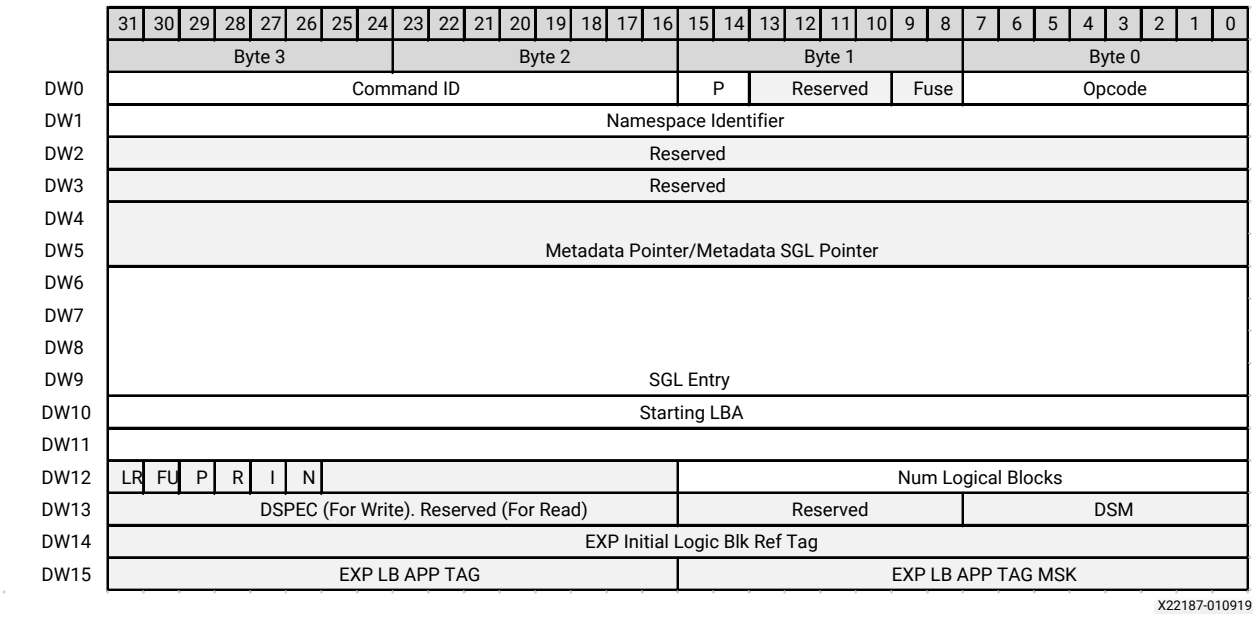


The AXI4 memory-mapped interface is used by the software (SW) to push SQ entries (SQEs) to the NVMe Host Accelerator while the AXI4-Lite interface is used for register accesses. The application hardware interface to NVMe Host Accelerator is through the AXI4-Stream, which the hardware application can use to push the SQEs.

The NVMe Host Accelerator exposes a memory mapped AXI interface for software and an AXI4-Stream interface for any hardware module interfacing with it. Despite the differences, the logical behavior of these interfaces is very similar. A FIFO type of an interface is exposed to the application for pushing the SQEs. If the QoS is not enabled (SW_QOS_EN = 0 for software and HW_QOS_EN = 0 for hardware), the NVMe Host Accelerator arbitrates the SQEs in a round-robin fashion across the available SQs for the chosen SSD. The arbitration is changed to another queue only if the current queue is full. Until an arbitrated queue is full, new requests are pushed into the same queue. If the QoS is enabled, this round-robin arbitration is disabled and the SQEs are pushed directly to the submission queue IDs provided by the application. This allows the application to apply specific QoS algorithms external to the IP. A similar FIFO type of interface is exposed for the completion queue entries (CQEs) coming from the drives as well. The header accompanying the CQE consists of SSD ID for the CQE. The information of about the CQID can be derived from the SQID information present in the CQE.

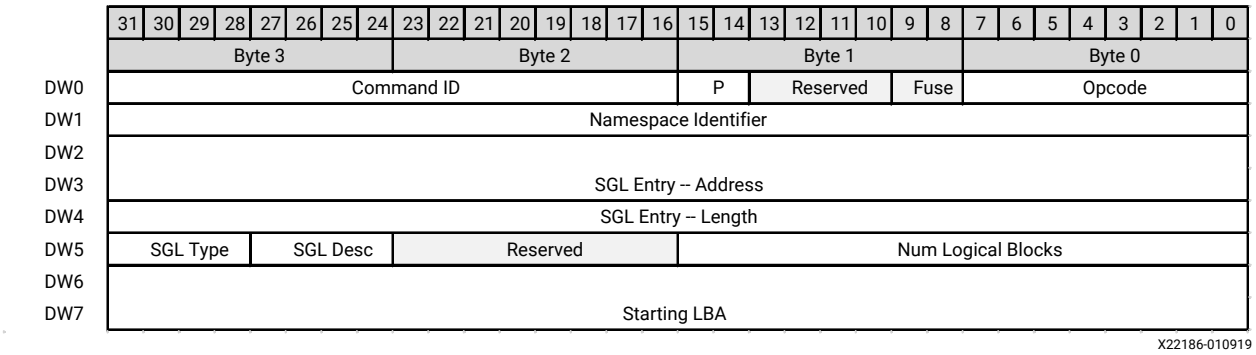
The application may choose to provide an optimized (partial) SQE (32-byte) or the full SQE (64-byte) to be pushed to the IP. The IP can be configured based on the parameter HW_FULL_CMD or SW_FULL_CMD for the hardware and the software interfaces respectively. The format of the complete 64 B SQE is shown in the following figure:

Figure 2: **SQE Format**



The SGL entry has three bytes which are reserved. This is used to optimize the instruction format so that a 32-byte SQE can be issued to the NVMe Host Accelerator. The 32-byte SQE is shown in the following figure:

Figure 3: **Partial SQ Command**



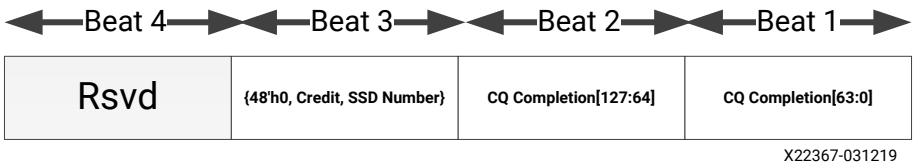
When a partial command is issued, the NVMe Host Accelerator fills in the unused fields with 0s before issuing commands to the NVMe drive. When partial command is enabled and the PRP mode of data transfer is selected, PRP list or multiple PRPs cannot be issued. Multi-page access is only possible using the SGL mode of data transfer when partial command is enabled.

The standard NVMe completion is 16 byte wide. The completion from the SSDs is appended with SSD ID and credit information and provided to the software on the memory mapped AXI slave. It takes eight cycles for one CQ completion when `AXI_MEM_DWIDTH = 32` and four cycles when `AXI_MEM_DWIDTH = 64`. The completion format to the memory interface is provided in the following figures:

Figure 4: **32-bit Interface Read Data**



Figure 5: **64-bit Interface Read Data**



The completion data for AXI streaming interface has two components. The CQ completion data is given as TDATA and SSD number and SQ credit is sent on TUSER as shown in the following figures:

Figure 6: **AXI Streaming Master TDATA**

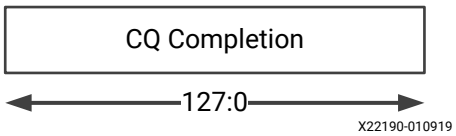
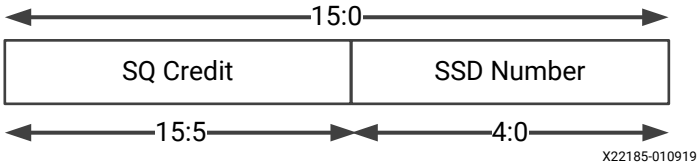


Figure 7: **AXI Streaming Master TUSER**



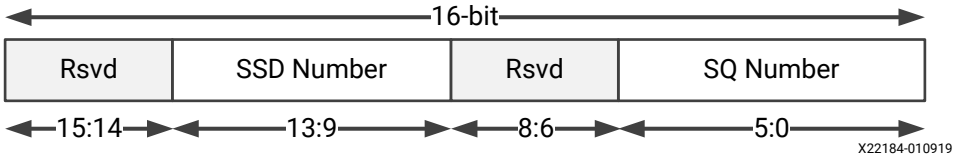
The SWAXI4 slave interface provides a FIFO type of interface for the software to post SQEs to the backend NVMe drives. The same interface is also used by the software to read the CQEs. The address of the incoming writes is used to divide the SSD ID and SQ ID for the SQEs. Writes to these locations amount to an SQE push to the NVMe Host Accelerator internal FIFO. If QoS is not enabled, the software should always write to the SQ0 register. The NVMe Host Accelerator internally arbitrates the commands to the relevant SSD SQs. It pops the entries from this FIFO, builds the NVMe command, and pushes them to the required SSD SQs. The reads to these addresses return the last written value. The following table provides the address map for the AXI4 slave interface. The NVMe Host Accelerator asserts the interrupt signal to inform the software of a new CQE.

Table 1: **SWAXI4 Slave Memory Map**

Address offset	SSD ID/SQ ID	Description
0x0_0000	SSD0/SQ0	
0x0_0040	SSD0/SQ1	
0x0_0080	SSD0/SQ2	
...	...	Space for up to 64 SQs
0x0_1000	SSD1/SQ0	
0x0_1040	SSD1/SQ2	
...	...	
0x1_F000	SSD31/SQ0	
0x1_F040	SSD31/SQ1	
...	...	
0x3_0000	SSDn/CQm	Read to this location provides the CQEs from the various backend SSDs. Only a single location is provided for CQEs of all SSDs. Each CQE is appended with a header that identifies the SSD ID for the CQE. This location shouldn't be read unless there are valid CQEs to be read out.

The AXI4-Stream slave interface provides a FIFO type of interface for the hardware application/ module to post SQEs to the backend drives. To write an SQE, "TVALID" is asserted by the hardware application. The NVMe Host Accelerator asserts "TREADY" to accept the submission. The SSD number and the SQ information is embedded in TUSER as shown in the following figure:

Figure 8: **TUSER Format for AXI4-Stream Command Submission**

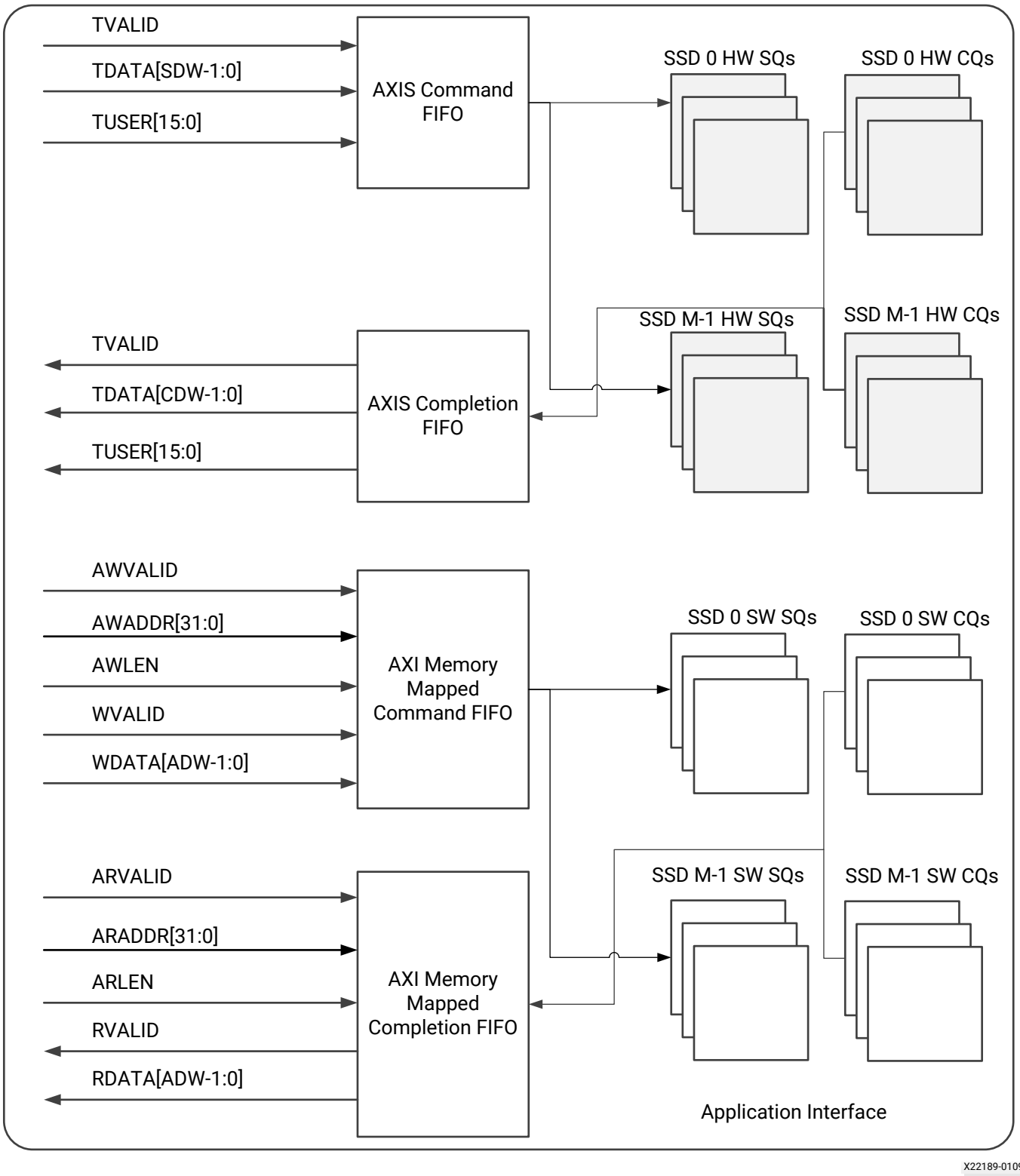


When the QoS parameter is not set, the SQ number is ignored by the NVMe Host Accelerator. When the QoS parameter is set, the command is pushed to the SQ number provided by the application. Submission queues per SSD is configurable using a parameter that can be set when generating the IP. The depth of each SQ is also parametrizable at IP generation. The NVMe Host Accelerator allows for a minimum of one queue and a maximum of four queues per SSD. The queues for each SSD can be assigned to a hardware application or a software application. This is decided at the time of IP generation. For example, if four queues are allocated for each SSD, for SSD0 two queues can be assigned to software, and the rest to the hardware application. For SSD1, the allocation between software and hardware application could be 1 and 3. It is also possible to allocate all queues to an SSD to either hardware or software application only. In case of an SSD having both software and hardware queues, the SSD ID allocation starts from software queues followed by hardware queues. For example in the earlier example for SSD0, SQ ID 1, 2, 3, and 4 are allocated to the software and SQ IDs 5 onwards are allocated to the hardware.

Each SQ has an associated CQ. Whenever a new SQ entry is posted, a request is submitted to the arbiter for a SQ tail doorbell update to the SSD. The arbiter may coalesce doorbell writes going to the SSD for improved efficiency. Similarly, when a CQ entry is read out internally from the application interface, a CQ head doorbell is written to the SSD interface. All these writes are arbitrated with a round-robin arbiter.

The NVMe Host Accelerator exposes a simple FIFO type interface to the application from queue pointer management. The application can push the SQEs through the interface exposed to them and the NVMe Host Accelerator manages placing these SQEs at the right pointer location and ringing the appropriate doorbell to SSD. The following figure shows the structure of the NVMe Host Accelerator FIFO interface.

Figure 9: **NVMe Host Accelerator FIFO Interface Structure**



In order to ensure that the application does not inundate the NVMe Host Accelerator with more requests than can be handled using the available SSD queues, a credit based mechanism is deployed. Based on the number of queues that are allocated to the software or hardware application per SSD, an initial credit is assumed. For example, if two queues of depth 32 are allocated to the software application for SSD0, the initial credit to the application would be 62 (queue is full when 31 entries are present). Each time the software submits an SQE to SSD0, this credit is reduced by 1. More credits are given back to the application through the CQE header. Each CQE popped by the software application provides an incremental credit to the application. This needs to be added to the total available credit for that SSD. The same credit flow is available for the hardware interface as well. In addition to the NVMe Host Accelerator provides the status of various queues in the status registers accessible using AXI4-Lite.

Performance and Resource Use

For full details about performance and resource use, visit the Performance and Resource Use web page.

Port Descriptions

The interfaces of NVMe Host Accelerator are given in the following table along with the expected connectivity.

Table 2: **NVMe Host Accelerator Interfaces**

Interface Name	Width	Interfacing Block	Purpose
AXI4-Lite	32	Software	Register access
SW_AXI4 Slave	32/64 depending on AXI_MEM_DWIDTH parameter	Software	To write SQEs and read CQEs by SW
HA_Interrupt	1	Software	Interrupt to indicate various scenarios
HW_AXI_Stream Slave	512 when HW_FULL_CMD = 1, else 256 with 16-bit TUSER	Hardware	To write SQEs by HW
HW_AXI_Stream Master	128-bit with 16-bit TUSER	Hardware	To read CQEs by HW
SSD_AXI4-Slave	128	SSD (via PCIe RP)	SSD interface to read SQEs and CQEs. The actual memories of SQEs and CQEs are stored in the NVMe Host Accelerator IP
SSD_AXI4-Master	32	SSD (via PCIe RP)	SSD interface to write SQ tail doorbell and CQ head doorbell

AXI4-Lite Interface Ports

Table 3: **AXI4-Lite Interface for Register Access**

Port Name	I/O	Clock	Description
clk_axi_lite	I		AXI4-Lite clock
rst_n_axi_lite	I		active-Low synchronous AXI4-Lite reset. Both resets (rst_n_core and rst_n_axi_lite) have to be asserted at the same time. They can be de-asserted independently.
s_axi_lite_*	-	clk_axi_lite	See the *Vivado Design Suite: AXI Reference Guide* (UG1037) for the description of AXI4 signals.

SW AXI Interface Ports

Table 4: **SW AXI Interface Ports**

Port Name	I/O	Clock	Description
clk_core	I		Core clock
rst_n_core	I		active-Low synchronous reset. Both resets (rst_n_core and rst_n_axi_lite) have to be asserted at the same time. They can be de-asserted independently.
s_axi_*	-	clk_core	See the *Vivado Design Suite: AXI Reference Guide* (UG1037) for the description of AXI4 signals.

HW AXI Interface Ports

Table 5: **HW AXI Interface Ports**

Port Name	I/O	Clock	Description
s_axis_tvalid	I	clk_core	Indicates that the master is driving a valid transfer
s_axis_tdata	I	clk_core	Data to be taken as SQE. Width is either 512 bits or 256 bits based on whether the C_HW_FULL_CMD are set or not
s_axis_tkeep	I	clk_core	All bits of TKEEP should be high
s_axis_tuser	I	clk_core	Indicates SSD and SQ number for a SQE
s_axis_tready	O	clk_core	Indicator that SQE has been accepted
m_axis_tvalid	O	clk_core	Asserted to push out CQEs to HW application
m_axis_tdata[127:0]	O	clk_core	CQE data
m_axis_tkeep	O	clk_core	All bits are set all the time
m_axis_tuser	O	clk_core	Indicates the SSD number and credits available for that queue
m_axis_tready	I	clk_core	Indicates that the CQE has been accepted by the master

SSD Master AXI Interface Ports for Doorbell Rings

Table 6: **SSD Master AXI Interface Ports for Doorbell Rings**

Port Name	I/O	Clock	Description
clk_core	I		Core clock
rst_n_core	I		active-Low synchronous reset
m_axi_ssd_*	-	clk_core	See the *Vivado Design Suite: AXI Reference Guide* (UG1037) for the description of AXI4 signals.

SSD Slave AXI Interface Ports for SQ Reads and CQ Writes

Table 7: **SSD Slave AXI Interface Ports for SQ Reads and CQ Writes**

Port Name	I/O	Clock	Description
clk_core	I		Core clock
rst_n_core	I		active-Low synchronous reset
s_axi_ssd_*	-	clk_core	See the *Vivado Design Suite: AXI Reference Guide* (UG1037) for the description of AXI4 signals.

Interrupts

Table 8: **AXI4-Lite Interface for Register Access**

Port Name	I/O	Clock	Description
hc_interrupt	-	clk_axi_lite	Single interrupt pin for all interrupts provided by NVMe Host Accelerator

Register Space

Table 9: **Register Address Space**

Address (hex)	Register
0x0000 + (n * 0x8)	PCIe base address lower
0x0004 + (n * 0x8)	PCIe base address upper
0x100	SSD0 SQ1 base address. This has to match to what the SSD sees as the first SQ address, which is defined by the create IO submission queue admin command
0x104	SSD0 CQ1 base address. This has to match to what the SSD sees as the first CQ address, which is defined by the create IO completion queue admin command
0x108 + (n * C_MAX_SQ * 16) + (m * 16)	SSDn SQm flush enable
0x10C + n * C_MAX_SQ * 16) + (m * 16)(	SSDn SQm flush status
0x100 + (n * C_MAX_SQ * 16) + (m * 16)	SSDn SQm base address
0x104 + (n * C_MAX_SQ * 16) + (m * 16)	SSDn CQm base address
0x8104	Interrupt enable register
0x8108	Interrupt status register
0x810C	SW available CQs
0x8110	Command error count
0x8114	Timeout register
0x8200 + (n * 16)	SSDn issued commands
0x8204 + (n * 16)	SSDn error completions
0x8208 + (n * 16)	SSDn completed commands
0x8400 + (n * C_MAX_SQ * 4) + (m * 4)	SQ and CQ FIFO status

SSDn PCIe Base Address Lower Register (0x0000 + (n * 8))

Table 10: **SSDn PCIe Base Address Lower Register (0x0000 + (n * 8))**

Bit	Default Value	Access Type	Description
31:0	0x0	RW	SSDn PCIe Base address lower 32-bits/64-bits

SSDn PCIe Base Address Upper Register (0x0004 + (n * 8))

Table 11: **SSDn PCIe Base Address Upper Register (0x0004 + (n * 8))**

Bit	Default Value	Access Type	Description
31:0	0x0	RW	SSD0 PCIe Base Address Upper 32-bits/64-bits

SSD0 SQ1 Base Address Register (0x0100)

Table 12: **SSD0 SQ1 Base Address Register (0x0100)**

Bit	Default Value	Access Type	Description
31:12	0x0	RW	SSD0 SQ1 Base address. This is the only SQ address that can be independently configured. All other SQ addresses are generated based on depth of SQ and base address as defined by this register.
11:0	0x0	RO	Reserved

SSD0 CQ1 Base Address Register (0x0104)

Table 13: **SSD0 CQ1 Base Address Register (0x0104)**

Bit	Default Value	Access Type	Description
31:12	0x0	RW	SSD0 CQ1 Base address. This is the only CQ address that can be independently configured. All other CQ addresses are generated based on depth of CQ and base address as defined by this register.
11:0	0x0	RO	Reserved

SSDn SQm Base Address Register (0x0100 + (m * C_MAX_SQ * 16) + (n * 16))

Table 14: **SSDn SQm Base Address Register (0x0100 + (m * C_MAX_SQ * 16) + (n * 16))**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	SSDn SQm Base address. This value is determined by the depth of previous SQ and base address of previous SQ.

SSDn CQm Base Address Register (0x0104 + (m * C_MAX_SQ * 16) + (n * 16))

Table 15: **SSDn CQm Base Address Register (0x0104 + (m * C_MAX_SQ * 16) + (n * 16))**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	SSDn CQm Base address. This value is determined by the depth of previous CQ and base address of previous CQ.

SSDn SQm Flush Register (0x0108 + (m * C_MAX_SQ * 16) + (n * 16))

Table 16: **SSDn CQm Base Address Register (0x0104 + (m * C_MAX_SQ * 16) + (n * 16))**

Bit	Default Value	Access Type	Description
0	0x0	RW	SSDn SQm Flush enable. Setting this bit results in clearing all the pending instructions to the corresponding SSDn/SQm. This bit must be cleared by the SW before restarting operation. The SW must ensure that appropriate action is also taken for the corresponding SSD. SW and HW application interfaces should ensure that after asserting SQ flush for a particular SQ, and no new SQEs for the SQ being flushed is pushed into NVMe Host Accelerator. For example, if SSD0/SQ0 is being flushed, no new commands should be pushed into the NVMe Host Accelerator that are targeted to SSD0/SQ0 till the flush done status bit is set to 1. NVMe Host Accelerator sets the flush done status bit of the targeted SQ to 1 once the flush operation is complete. Once this is done, the flush bit must be reset by the SW to ensure further operation. In case where there is no QoS enabled, it is the responsibility of the SW to set the flush bit for all the SQs of a particular SSD if there is an error condition with the SSD. Any pending CQ doorbells are discarded if this bit is set.
31:1	0x0	RO	Reserved

SSDn SQm Flush Done Status Register (0x010C + (m * C_MAX_SQ * 16) + (n * 16))

Table 17: **SSDn SQm Flush Done Status Register (0x010C + (m * C_MAX_SQ * 16) + (n * 16))**

Bit	Default Value	Access Type	Description
0	0x0	RO	SSDn SQm Flush done status. When the corresponding bit for flush is set, all pending SQs are flushed out and pointers are reset. All pending CQ doorbells are discarded. Once the CQ data that is already pushed into AXI buffers is also sent out on AXI, the done status for the flush bit is set. This ensures that there are no pending or stale data. The application interface should reset the flush bit once done status is set.
31:1	0x0	RO	Reserved

Interrupt Enable Register (0x8104)

Table 18: **Interrupt Enable Register (0x8104)**

Bit	Default Value	Access Type	Description
0	0x0	RW	CQ Available interrupt enable
1	0x0	RW	SW AXI/HW AXIS error address interrupt enable
2	0x0	RW	SW AXI credit error interrupt enable
3	0x0	RW	SW AXI read without CQs interrupt enable
4	0x0	RW	HW AXI write without available credits interrupt enable
5	0x0	RW	SSD timeout interrupt enable
31:6	0x0	RO	Reserved

Interrupt Status Register (0x8108)

Table 19: **Interrupt Status Register (0x8108)**

Bit	Default Value	Access Type	Description
0	0x0	RO	CQ Available interrupt status.
1	0x0	RO	Software AXI/Hardware AXIS error interrupt status. When a non existing SSD/SQ combination is accessed on software AXI or hardware AXIS, this bit is set to indicate that the command is addressed to an undefined SSD/SQ combination and the corresponding SQE is not processed.
2	0x0	RO	Software AXI credit error interrupt status. When there is a SQE even though there are not enough credits for software queues, this bit is set.
3	0x0	RO	Software AXI read without CQs interrupt status. When there are no available CQEs to read but there was a CQ read by the software interface this bit is set.
4	0x0	RO	Hardware AXI write without available credits interrupt status. When there is a SQE from a hardware interface to a SSD/SQ without available credits, this bit is set. The corresponding SQE is discarded.
5	0x0	RO	SSD timeout interrupt status. When there is no CQ entry for a corresponding SQE within a time defined by timeout register, this bit is set.
31:6	0x0	RO	Reserved

If CQ available interrupt enable bit is set for software, an interrupt is raised if there are non-zero number of CQs available to read on the software interface. If an error is detected in the NVMe Host Accelerator AXI input interface (either hardware or software interface), and AXI command error interrupt is enabled, an interrupt is raised.

When an interrupt is raised for software to read CQ, the following procedure must be followed by software to read the available CQs correctly:

- Read CQ available count register. Once read, the CQ available register should not be read again till the next interrupt is set.

- Process all CQ entries by reading less than or equal to the number of available CQs. If the software decides to read lesser number of CQs than are available, it is up to the software to treat the next available CQs as an incremental number. For example: say there are eight available CQs and the software decides to read six of them. If the next time, the CQ available reads as three, it means the software has a total of five to read and process (two from previous unread CQs and three from the current interrupt)
- Clear interrupt status by writing a 1 to bit 0 of the interrupt status register.

  Note: If C_EN_SW_IF is set to 1 (software interface is enabled), CQ available interrupt should always be enabled. A polling style of reading CQ available count register is not supported.

CQ Available Count Register (0x810C)

Table 20: **CQ Available Count Register (0x810C)**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	This register provides the count of CQs available for the SW to read. The SW is expected to read this register when it receives a CQ available interrupt to know the number of CQs that may be popped from the SW interface.

Command Error Count Register (0x8110)

Table 21: **Command Error Count Register (0x8110)**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	Whenever the NVMe Host Accelerator encounters any error in the commands issued by SW or HW interfaces, the error command count is incremented. In case of a simultaneous error on both interfaces, the count is incremented by 2. The count wraps after the maximum value is reached.

Timeout Register (0x8114)

Table 22: **Timeout Register (0x8114)**

Bit	Default Value	Access Type	Description
31:0	0x9C4	RW	A timeout value is defined by this register is used to set an interrupt and status bit under the following conditions: • SQ is non-empty • No completion is received on the corresponding CQ for a period of cycles as defined by this register • Timeout value is programmed with a non-zero value After every interval as defined by this register, all timed out SQs have the bit set in the status register. An interrupt is also raised if enabled.

SSDn Issued Commands Count Register (0x8200 + (n * 16))

Table 23: **SSDn Issued Commands Count Register (0x8200 + (n * 16))**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	This counter provides the total number of commands issued from all SQs to the SSD. The counters wrap around after the maximum value is reached.

SSDn Error Completion Commands Count Register (0x8204 + (n * 16))

Table 24: **SSDn Error Completion Commands Count Register (0x8204 + (n * 16))**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	This counter provides the total number of commands that completed with error status. The counters wrap around after the maximum value is reached.

SSDn Completion Commands Count Register (0x8208 + (n * 16))

Table 25: **SSDn Completion Commands Count Register (0x8208 + (n * 16))**

Bit	Default Value	Access Type	Description
31:0	0x0	RO	This counter provides the total number of commands that completed. The counters wrap around after the maximum value is reached.

SSDn/SQmCQm Status Register (0x8400 + (n * C_MAX_SQ *4) + (m * 4))

Table 26: **SSDn/SQmCQm Status Register (0x8400 + (n * C_MAX_SQ *4) + (m * 4))**

Bit	Default Value	Access Type	Description
0	0x0	RO	SSDn/SQm Full
1	0x1	RO	SSDn/SQm Empty
2	0x0	RO	SSDn/CQm Full
3	0x1	RO	SSDn/CQm Empty
12:4	HWDependent	RO	SSDn/SQm available locations. Initial value depends on the depth configured.

Table 26: **SSDn/SQmCQm Status Register (0x8400 + (n * C_MAX_SQ *4) + (m * 4))** *(cont'd)*

Bit	Default Value	Access Type	Description
13	0x0	RO	SSDn/SQmCQm response timeout

Related Information

NVMe Host Accelerator Programming Sequence

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

Use the Example Design

Each instance of the NVMe Host Accelerator core created by the Vivado design tool is delivered with an example design that can be implemented in a device and then simulated. This design can be used as a starting point for your own design or can be used to sanity-check your application in the event of difficulty. See the Example Design content for information about using and customizing the example designs for the core.

Registering Signals

To simplify timing and increase system performance in a programmable device design, keep all inputs and outputs registered between the user application and the core. This means that all inputs and outputs from the user application should come from, or connect to, a flip-flop. While registering signals might not be possible for all paths, it simplifies timing analysis and makes it easier for the AMD tools to place and route the design.

Recognize Timing Critical Signals

The constraints provided with the example design identify the critical signals and timing constraints that should be applied.

Make Only Allowed Modifications

You should not modify the core. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the core can only be made by selecting the options in the customization IP dialog box when the core is generated.

Reference Design

This is a fixed configuration comprising of four SSDs with four SQs each. Two SQs are assigned to the hardware and two are assigned to the software interface. There are other supporting IPs in AMD Vivado™ Design Suite that are required to build the reference design. For more information about reference design, see the reference design files located here.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
- *Vivado Design Suite User Guide: Designing with IP* (UG896)
- *Vivado Design Suite User Guide: Getting Started* (UG910)
- *Vivado Design Suite User Guide: Logic Simulation* (UG900)

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

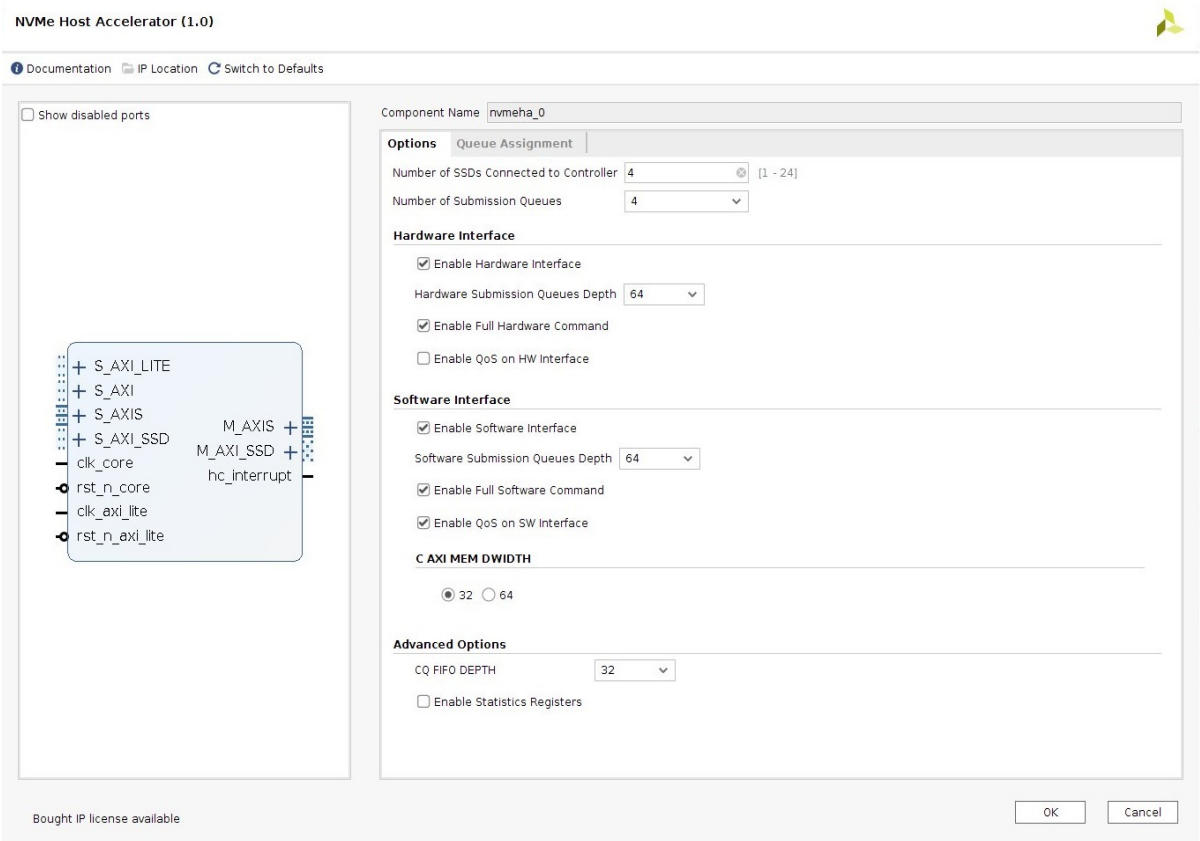
For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) and the *Vivado Design Suite User Guide: Getting Started* (UG910).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

To locate this IP in the catalog, navigate to **IP Catalog → Storage IP → NVMe Host Accelerator**.

You can configure the IP for a different number of SSDs with hardware and software interfaces that have different depth of SQ as shown in the following figure.

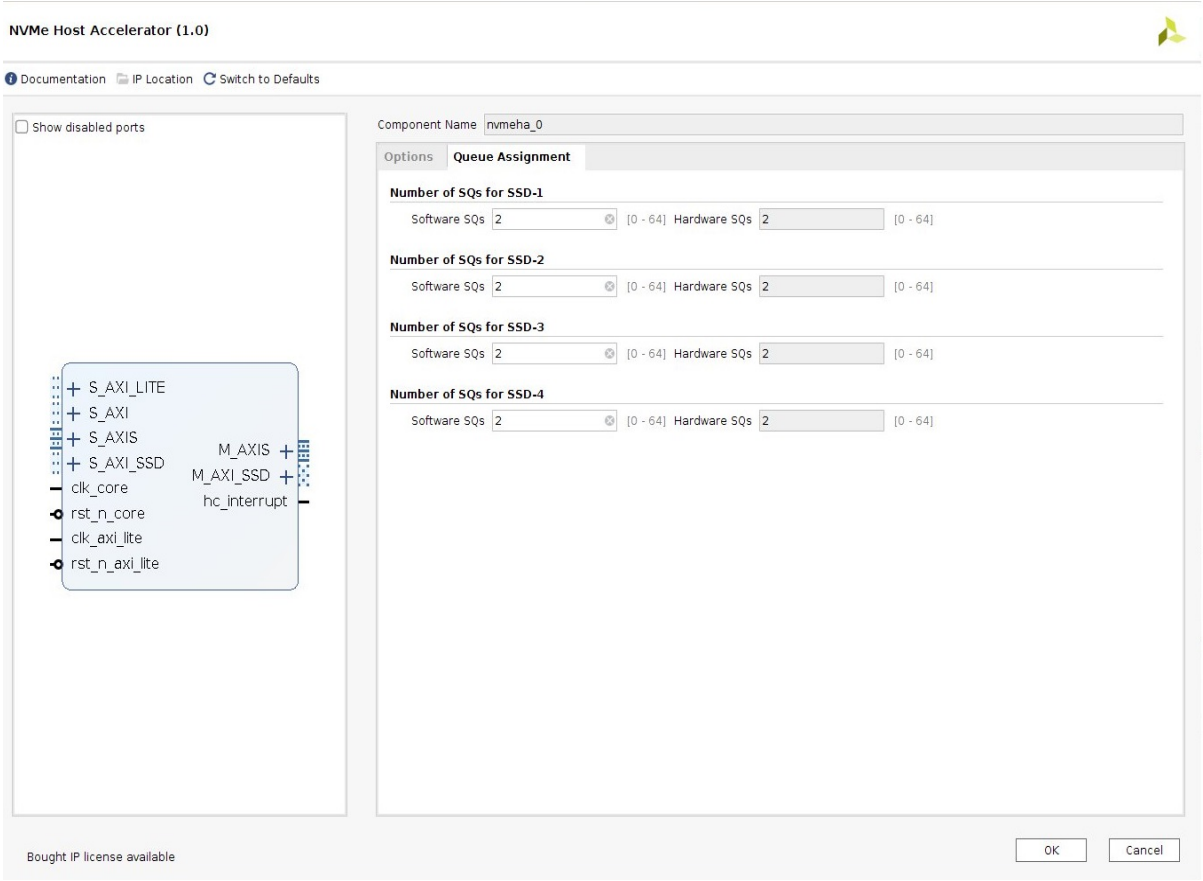
Figure 10: **IP Configuration**



The SQ FIFO depth defines the depth of the FIFO in each interface for storage of SQ command before actually being transferred into the SQ buffer in NVMe Host Accelerator. The CQ FIFO depth defines the depth of the CQ FIFO that stores the CQ response before issuing to AXI interface.

The number of SQ/CQ pair can be configured per SSD as shown in the following figure.

Figure 11: **Per SSD number of SQs configuration**



User Parameters

The following table shows the relationship between the fields in the AMD Vivado™ IDE and the user parameters (which can be viewed in the Tcl Console).

Table 27: **User Parameters**

Vivado IDE Parameter/ Value[1]	User Parameter/Value	Default Value
C_MAX_SSD	1-24	Number of SSDs Default: 4
C_MAX_SQ	1, 2, 4	Number of SQs for each SSD Default: 4
C_SQ_DEPTH_HW	32, 64	SQ depth of all SQs assigned to the hardware Default: 64
C_SQ_DEPTH_SW	32, 64	SQ depth of all SQs assigned to the software Default: 64

Table 27: **User Parameters** *(cont'd)*

Vivado IDE Parameter/ Value[1]	User Parameter/Value	Default Value
C_NUM_SQ_HW{C_MAX_SSD - 1:0]	0-4	Number of SQs per SSD for the hardware (when set to 0, C_EN_HW_IF automatically is set to 0) Default: 2
C_NUM_SQ_SW[C_MAX_SSD - 1:0]	0-4	Number of SQs per SSD for the software (when set to 0, C_EN_SW_IF is automatically set to 0) Default: 2
C_EN_HW_IF	Enable/Disable	Enable the hardware interface Default: Enable
HW_QOS_EN	Enable/Disable	Enable QoS for hardware AXI-S interface Default: Disable
C_EN_SW_IF	Enable/Disable	Enable the software interface Default: Enable
SW_QOS_EN	Enable/Disable	Enable QoS for software AXIMM interface Default: Enable
AXI_MEM_DWIDTH	32/64	Software AXIMM interface data width Default: 32
CQ_FIFO_DEPTH	16-64	CQ FIFO depth. A higher value of this parameter ensures a lesser back pressure to SSD Default: 16
C_HW_FULL_CMD	Enable/Disable	Hardware full command enable. When enabled, streaming data width is 512- bits. When disabled, streaming data width is 256-bits.
C_SW_FULL_CMD	Enable/Disable	Software full command enable. When enabled, memory mapped data width is 512-bits. When disabled, memory mapped data width is 256-bits Default: Enable
STATISTICS_EN	Enable/Disable	Enable statistics registers Default: Disable

Notes:

1. Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

NVMe Host Accelerator Programming Sequence

Based on the PCIe address for SSD, program the same base address into the PCIe base address lower (0x0000 + n * 0x08). For example, if the PCIe address for SSD is `0xB000_0000`, program the same value in the PCIe base address lower register. The parameter C_MAX_SSD defines the maximum number of SSDs to be connected to the NVMeHA. There are C_MAX_SSD number of PCIe base address lower registers and all of them must be programmed with the corresponding PCIe address.

The parameter C_MAX_SQ defines how many SQs are created in the SSD. All the SSDs have the same number of SQs. You can enable the SQ base address of SSD0, SQ1 (0x100), and CQ base address of SSD0 (0x104). For example, if the SQ depths of all queues are 64 and there are four queues, then the following is true:

- Base address of SQ 1 of SSD0 is `0xA000_0000`.
- SQ 2 base address of SQ2 of SSD0 is `0xA000_1000`.
- SQ3 base address is `0xA000_2000`.
- SQ4 base address is `0xA000_3000`.
- The base addresses of subsequent SSDs SQs is determined by the base address of SSD0s SQ1:
  - SQ1 of SSD1 is `0xA000_4000`.
  - SQ2 of SSD1 is `0xA000_5000`.
  - SQ3 of SSD1 is `0xA000_6000`.
  - SQ4 of SSD1 is `0xA000_7000`.

The same addressing scheme is applied for other CQs as well. The CQ1 of SSD0 can be given any value (that is 4 KB aligned) and the other CQ address of SSD0 and subsequent SSDs is determined as mentioned above. If there are software queues enabled by setting C_EN_SW_IF and configuring C_NUM_SQ_SW, then the software SQs must be addresses as mentioned in Table 1: SWAXI4 Slave Memory Map.

When software SQs have CQ completions to be read out interrupt is set (0x8104's corresponding bits have to be set to generate an interrupt). To know how many CQs are available to process, read 0x810C (software available CQs). Hardware assigned SQs do not generate an interrupt. Instead they generate a TVALID on the hardware AXI interface.

Constraining the Core

Required Constraints

Table 28: **False Path Constraints**

From Clock	To Clock
clk_core	clk_axi_lite
clk_axi_lite	clk_core

The clocks `clk_core` and `clk_axi_lite` are asynchronous to each other. A false path constraint should be set to ensure that the design works as intended.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

Table 29: **Clock Frequencies**

Clock Name	Maximum Frequency
clk_core	200/250 MHz[1]
clk_axi_lite	100 MHz

Notes:

1. If the total number of queues in the system are less than or equal to 32, then `clk_core` can be constrained at 250 MHz. If the total number of queues in the system are greater than 32, then `clk_core` must be constrained at 200 MHz.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

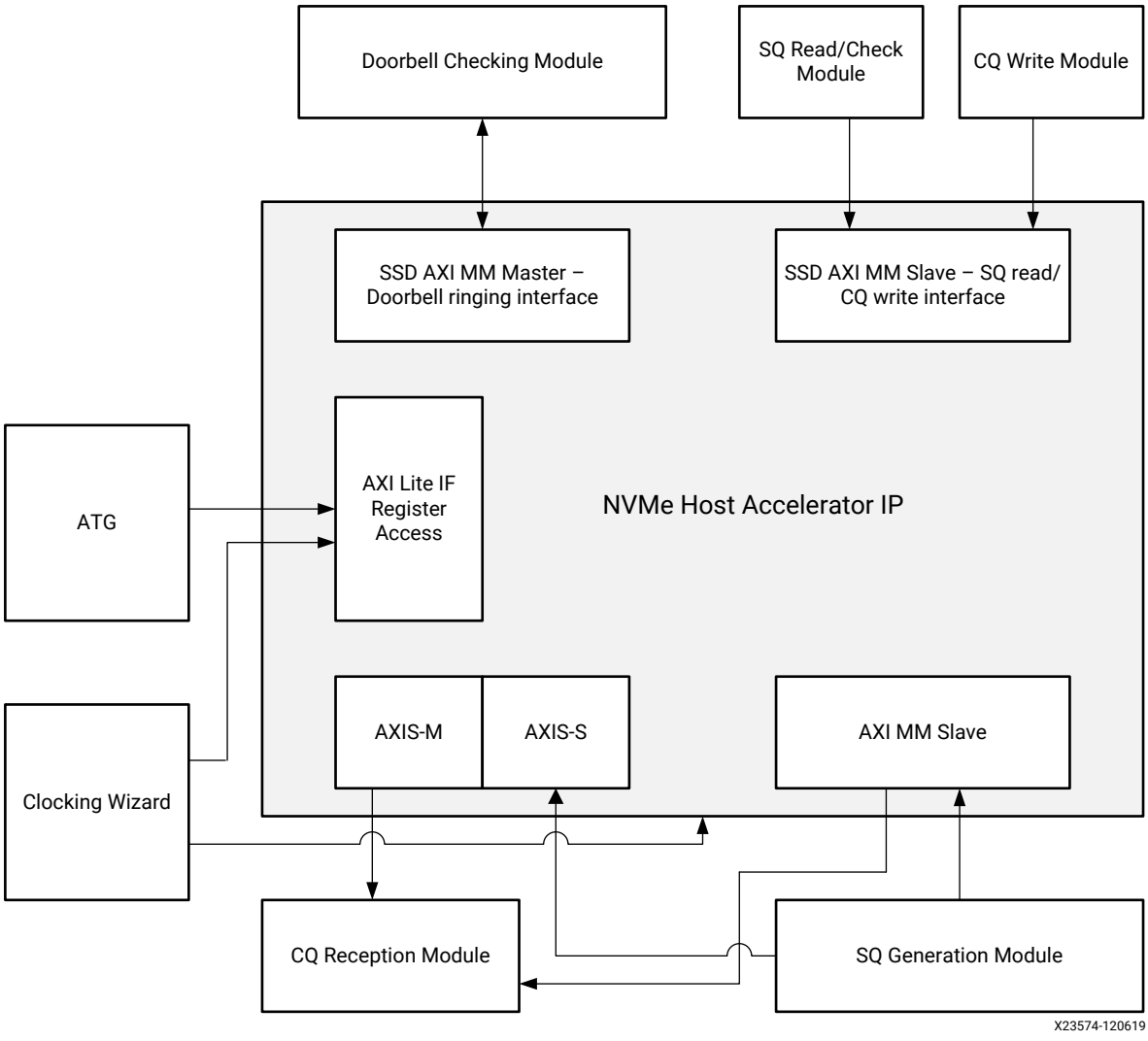
Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* (UG896).

Example Design

The example design for the NVMe Host Accelerator can be used to understand the IP behavior. The example design interface is shown in the following figure.

Figure 12: **Example Design Architecture**



Following are the blocks in the example design:

- **ATG:** To generate AXI4-Lite traffic to configure the IP

- **Clocking Wizard:** Required to generate clocks for AXI4-Lite interface and rest of the logic.
- **SQ Generator:** SQ generator block generates SQs for the enabled interfaces of all SSDs.
- **Doorbell Check Module:** The NVMe Host Accelerator generates doorbells for SQ tail and CQ head to the SSD. This module checks if the doorbells are rung or not
- **SQ Check:** When all doorbells are received for an SSD, SQs are checked for correctness by this module.
- **CQ Write:** CQs are written to the corresponding queues and doorbell check module checks that all the CQ completions have a corresponding head doorbell.
- **CQ Reception Module:** Checks for the CQs received.

When a CQ check is completed for an SSD, check starts for the next SSD. The process is repeated till all SSDs are covered. Reference design files are also provided via lounge (registration required). The reference design uses a traffic generator to pump traffic to the NVMe Host Accelerator on the AXI4-Slave interface as shown in the following figure.

Figure 13: **Reference Design**

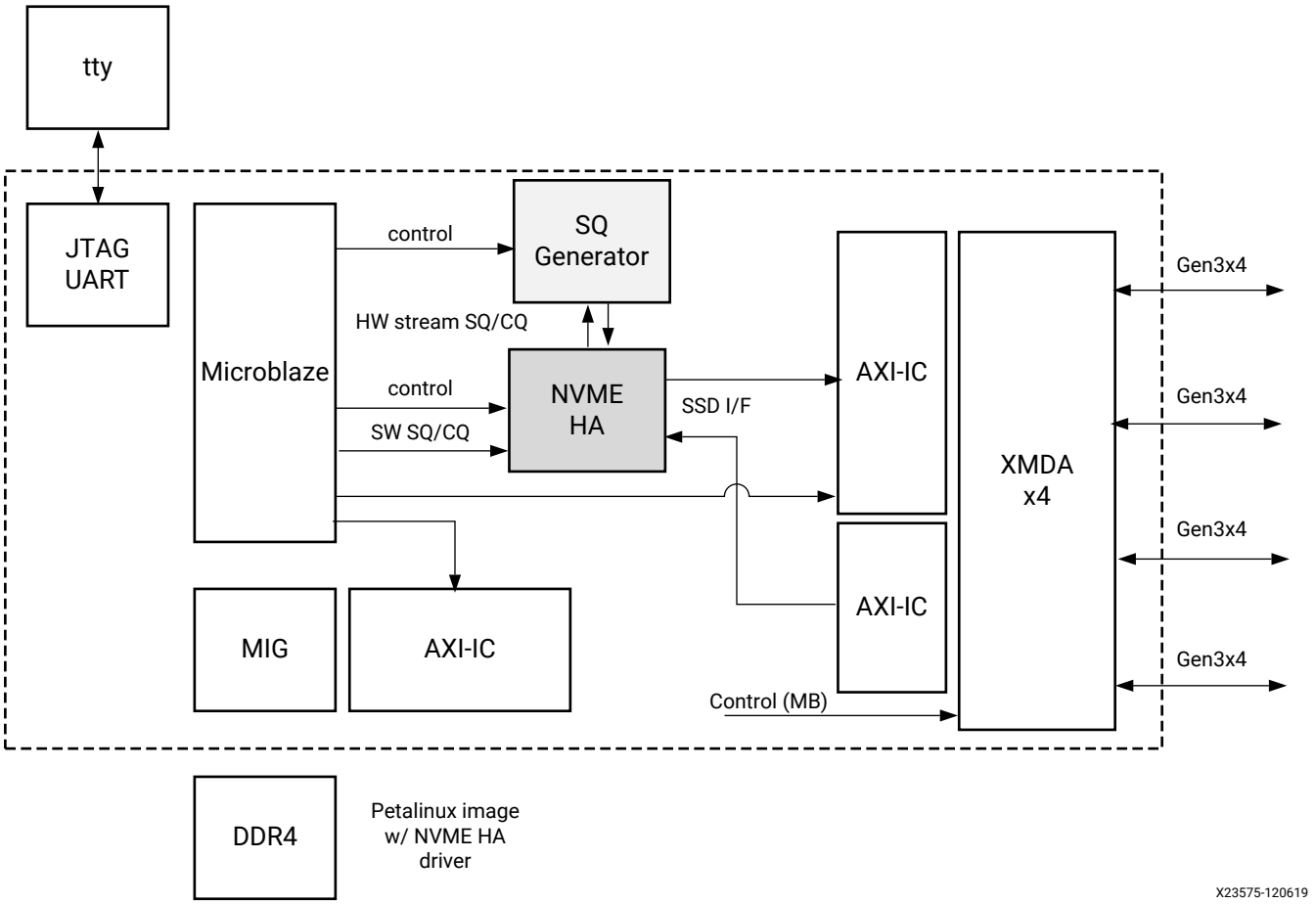


Table 30: **NVMe IP Slave Memory Size Calculator**

Parameter	Description and Value	Buffer Name	Buffer Size Equation (Bits)	Buffer Size (KB)	Block RAM Count	Associated Slave Interface	Description
C_MAX_SSD	Number of SSDs = 4	-	-	-	-	-	-
C_MAX_SQ	Number of SQs = 8	SW SQ/CQ Buffers	-	-	-	SW_AXI4 Slave	Software interfaced AXI4 slave. This interface is used to write submission queues of host controller and read from CQs.
C_SQ_DEPTH_HW	HW queues SQ depth = 128	HW SQ Buffers	C_MAX_SSD * C_MAX_SQ * (max (C_SQ_DEPTH_HW, C_SQ_DEPTH_SW)) * 512	256	8	SSD_AXI4-Slave	-
C_SQ_DEPTH_SW	SW queues SQ depth = 32	SW SQ Buffers	C_MAX_SSD * C_MAX_SQ * 64 * 128	32	1	SSD_AXI4-Slave	-
C_NUM_SQ_HW[C_MAX_SSD -1:0]	Number of queues assigned to HW per SSD	-	-	-	-	-	-
C_NUM_SQ_SW[C_MAX_SSD -1:0]	Number of queues assigned to SW per SSD	Statistics Registers	(C_MAX_SSD * C_MAX_SQ * 32) + 3 * (C_MAX_SSD)	0.126464844	1	AXI4-Lite	-
					Block RAM total = 10		

Appendix A

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)

IMPORTANT! *IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.*

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the Support web page contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The Community Forums are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the Support web page or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the Downloads page. For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main Support web page. To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Technical Support

AMD Adaptive Computing provides technical support on the Community Forums for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the Community Forums.

Debug Tools

There are many tools available to address NVMe Host Accelerator design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* (UG908).

Appendix B

Additional Resources and Legal Notices

Finding Additional Documentation

Documentation Portal

The AMD Adaptive Computing Documentation Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Documentation Portal, go to https://docs.xilinx.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the Design Hubs webpage.

***Note*:** For more information on DocNav, see the Documentation Navigator webpage.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this product guide:

1. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
2. *Vivado Design Suite User Guide: Designing with IP* (UG896)
3. *Vivado Design Suite User Guide: Getting Started* (UG910)
4. *Vivado Design Suite User Guide: Logic Simulation* (UG900)
5. *Vivado Design Suite User Guide: Programming and Debugging* (UG908)
6. *Vivado Design Suite: AXI Reference Guide* (UG1037)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
05/10/2023 Version 1.0	
Chapter 3: Product Specification	• Updated PRP and Queue text. • Updated figure: TUSER Format for AXI4-Stream Command Submission.
User Parameters	Updated values.
08/06/2021 Version 1.0	
General Updates	Corrected typographical errors.
06/03/2020 Version 1.0	
NVMe Host Accelerator Programming Sequence	Added new section.
12/19/2019 Version 1.0	
Chapter 6: Example Design	Added Example Design chapter.
06/04/2019 Version 1.0	
Initial release.	N/A

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2019-2023 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, Versal, Virtex, Vivado, and combinations thereof are trademarks of Advanced Micro Devices, Inc. AMBA, AMBA Designer, Arm, ARM1176JZ-S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the US and/or elsewhere. PCI, PCIe, and PCI Express are trademarks of PCI-SIG and used under license. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.